

第一周报告：项目启动阶段

报告周期：第 1-3 天

基本信息

团队信息：

- 团队名称：农业大棚监控系统
- 团队编号：二班九组（CG3-AGMS）
- 项目名称：农业大棚监控系统
- 提交日期：2026-05-31

团队成员信息：

姓名	学号	负责技术栈	主要职责
陈子建	2023210581	Node.js + Express	UniApp 开发
陈浩	2023210579	ArkTS + HarmonyOS SDK	鸿蒙应用开发
王秀妹	2023210631	Android Studio + Kotlin + Android	安卓开发
匡林	2023210602	STM32 + C/C++	嵌入式开发
李韶涵	2023210605	React.js + TypeScript	前端开发

第一天任务完成情况

☒ 团队组建完成

1. 团队成员确定

- ☒ 5 人团队组建完成
- ☒ 成员技能互补性分析完成
- ☒ 团队章程/协作规范制定完成

团队技术栈分布：

技术栈选择（在对应技术栈前打√）

- ☒ Android 原生开发（Java/Kotlin） - 成员：王秀妹（2023210631）
- ☒ React Native（JavaScript/TypeScript） - 成员：李韶涵（2023210605）
- ☒ 鸿蒙多端开发（ArkTS/ArkUI） - 成员：陈浩（2023210579）

☒ UniApp 开发（语言：Node.js） - 成员：陈子建（2023210581）

☒ 传感器开发（STM32） - 成员：匡林（2023210602）

注：Android、Flutter、React Native、鸿蒙、微信小程序为必选技术栈，每位同学至少掌握其中一个，确保跨平台整合的多样性。

2. 技术栈分工明确

成员姓名	技术栈	预期技术深度	学习基础	挑战难度评估
陈子建	Node.js + Express	熟练掌握 RESTful API 设计	有 Web 开发基础	中
陈浩	ArkTS + HarmonyOS SDK	熟练掌握鸿蒙应用开发	有移动开发经验	中
王秀妹	Android Studio + Kotlin + Android	熟练掌握安卓应用开发	有 Java 语言基础	中
匡林	STM32 + C/C++	掌握嵌入式硬件开发	有 C 语言基础	中
李韶涵	React.js + TypeScript	掌握前端框架开发	有 Web 开发基础	中

3. 团队负责人选定

- 项目经理：陈浩（负责整体协调、进度管理）
- 技术负责人：陈子建（负责技术方案、架构设计）
- 质量负责人：李韶涵（负责代码审查、测试管理）

4. 沟通方式建立

- 日常沟通工具：☒ qq 群
- 文档协作平台：☒ 腾讯文档
- 代码管理平台：☒ GitHub
- 每日站会时间：每天 20:00（时长：15 分钟）

☒ 项目选择确定

1. 选择的项目主题：农业大棚监控系统

2. 项目需求理解

核心功能列表：

- 1. 分布式传感器网络：实时采集大棚内温度、湿度、光照等环境数据
- 2. 设备联动控制：支持灌溉、通风、加热、降温等设备的智能控制
- 3. AI 种植策略优化：基于环境数据提供智能种植建议
- 4. AI 智能语音对话：支持语音交互的智能农业助手
- 5. 多端数据展示：跨平台实时监控大棚状态

创新点/特色功能：

- AI 语音助手功能：语音转文字+大模型问答+文字转语音
- 智能环境调节：基于 AI 分析自动调节设备
- 病虫害预警：结合环境数据提供病虫害预防建议

3. 技术挑战评估

技术挑战	难度等级	预计解决方案	风险评估
应用开发	高	学习语法和搭建框架	中
AI 大模型接入	中	使用豆包 API 进行接入	低
传感器数据采集	中	使用 Python+MQTT 协议	低
硬件设备控制	高	STM32+继电器控制	中

4. 实现可行性分析

技术可行性：

- ☒ 所需技术栈团队成员熟悉或可快速学习
- ☒ 第三方库/服务调研完成且可获取
- ☒ 开发时间充足（15 天）
- ☒ 技术难点有备选方案

资源可行性：

- ☒ 开发设备齐全（测试机型、开发电脑）
- ☒ 开发工具和 SDK 已获取
- ☒ 第三方服务（API、云服务）可用
- ☒ 网络环境满足开发需求

风险评估：

风险等级：☒ 中等风险

主要风险点：

- 1. 应用开发学习曲线较陡
- 2. AI 大模型 API 调用可能不稳定
- 3. 硬件设备调试需要实际设备

应对措施：

- 1. 提前学习语法，边学边做
- 2. 实现本地 AI 降级方案
- 3. 使用模拟器进行前期开发

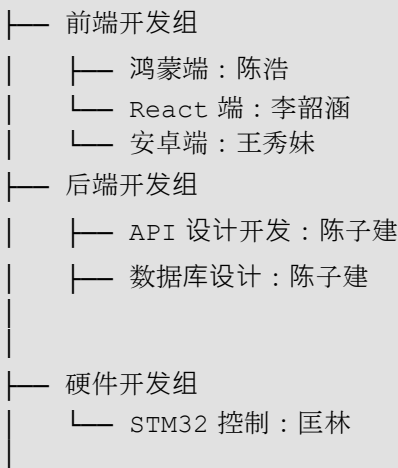
☒ 初步分工安排

1. 各成员技术栈确定

成员	开发平台	主要模块	预期工作量	技术难点
陈子建	后端服务	API 设计、数据库设计、设备控制逻辑	20%	MQTT 消息队列
陈浩	鸿蒙应用	界面开发、AI 语音、设备控制 UI	25%	ArkTS 状态管理
王秀妹	Python 服务	传感器数据采集、MQTT 通信	15%	传感器驱动
匡林	嵌入式	设备控制、继电器驱动	15%	硬件调试
李韶涵	React Web	数据可视化、管理后台	15%	前端框架

2. 核心职责划分

职责分工图



- └─ 整合测试组
 - └─ 跨平台整合：全员
 - └─ 功能测试：李韶涵
 - └─ AI 功能测试：陈浩

3. 协作方式约定

- 代码审查机制：每次 PR 需至少 1 人审核
- 代码提交规范：feat/fix/docs 等规范
- 分支管理策略：feature/xxx 格式
- 冲突解决机制：技术负责人仲裁

4. 进度管理机制

- 进度追踪工具：☒ Trello ☒ 腾讯文档
- 进度汇报周期：☒ 每日
- 里程碑设置：
 - 第 3 天：完成架构设计、项目初始化
 - 第 7 天：完成核心功能开发
 - 第 12 天：完成系统整合测试
 - 第 15 天：完成项目交付

第二天任务完成情况

☒ 系统架构设计

1. 整体架构图

系统架构：

移动端层

- └─ 鸿蒙端 (ArkTS)
 - | └─ UI 层：ArkUI 声明式
 - | └─ 业务层：Ability/Service
 - | └─ 数据层：Preferences/SQLite
 - | └─ 分布式能力：分布式任务调度
- └─ React Web 端
 - | └─ UI 层：React 组件
 - | └─ 业务层：React Hooks
 - | └─ 数据层：Context API

```
|
|   └─ 状态管理：Redux
└─ 微信小程序端
    └─ UI 层：WXML/WXSS
    └─ 业务层：JS 逻辑
    └─ 数据层：Storage
```

服务端层

```
└─ API 网关：Node.js Express
└─ 业务服务：设备控制 API
└─ 数据服务：传感器数据 API
└─ 第三方集成：豆包 AI API
```

硬件层

```
└─ STM32 主控
└─ 温湿度传感器
└─ 光照传感器
└─ 继电器控制模块
```

数据层

```
└─ 数据库：MySQL
└─ 缓存：Redis
└─ 文件存储：本地存储
```

架构设计说明：系统采用分层架构设计，前端通过 HTTP/MQTT 与后端通信，后端协调硬件层和控制层，实现大棚环境的智能监控与管理。

2. 技术栈集成方案

各平台技术选型：

平台	开发语言	UI 框架	网络库	本地存储	状态管理
Android 原生	Java	XML 布局	Retrofit	Room	ViewModel
iOS 原生	Swift	UIKit	URLSession	CoreData	Combine
Flutter	Dart	Material	dio	Hive	BLoC
React Native	TypeScript	RN 组件	axios	AsyncStorage	Redux
微信小程序	JavaScript	WXML/WXSS	wx.request	wx.storage	全局状态
鸿蒙 ArkTS	ArkTS	ArkUI 声明式	@ohos.net.http	首选项	@State 装饰器
后端服务	Node.js	Express	内置 http	MySQL	-

技术选型说明：

- Android 推荐：Kotlin + Jetpack Compose + Retrofit + Room + ViewModel
- iOS 推荐：Swift + SwiftUI + URLSession + CoreData + Combine

- Flutter 推荐: Dart + Material + dio + sqflite + Provider/Bloc
- React Native 推荐: TypeScript + RN 组件 + axios + AsyncStorage + Redux
- 鸿蒙推荐: ArkTS + ArkUI + @ohos.net.http + 首选项 + 装饰器状态管理

跨平台通信方案:

- 数据同步机制: HTTP 轮询 + MQTT 订阅
- 实时通信方案: MQTT 消息队列
- 离线存储策略: 本地缓存优先
- 冲突解决机制: 服务端数据权威

3. 数据库设计

数据库选型: ☒ MySQL

核心数据表设计:

表 1: **device** (设备表)

字段列表:

```
- id: INT PRIMARY KEY AUTO_INCREMENT
- device_name: VARCHAR(100) NOT NULL
- device_type: VARCHAR(50) NOT NULL
- status: TINYINT DEFAULT 0
- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- updated_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
```

表 2: **sensor_data** (传感器数据表)

字段列表:

```
- id: INT PRIMARY KEY AUTO_INCREMENT
- device_id: INT NOT NULL
- temperature: DECIMAL(5,2)
- humidity: DECIMAL(5,2)
- light_level: INT
- recorded_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- FOREIGN KEY (device_id) REFERENCES device(id)
```

表 3: **control_log** (控制日志表)

字段列表:

```
- id: INT PRIMARY KEY AUTO_INCREMENT
- device_id: INT NOT NULL
- action: VARCHAR(50) NOT NULL
- result: TINYINT
- operated_by: VARCHAR(100)
- operated_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

ER 图说明: 设备表与传感器数据表为一对多关系, 设备表与控制日志表为一对多关系。

4. API 接口规范

接口设计原则:

- ☒ RESTful 风格
- ☐ GraphQL
- 核心接口列表:

接口路径	请求方法	功能说明	请求参数	响应格式
/api/devices	GET	获取设备列表	-	JSON
/api/devices/:id	GET	获取设备详情	deviceId	JSON
/api/devices/:id/control	POST	控制设备	deviceId, action	JSON
/api/sensors/latest	GET	获取最新传感器数据	deviceId	JSON
/api/sensors/history	GET	获取历史数据	deviceId, startTime, endTime	JSON
/api/ai/advice	POST	获取 AI 建议	temperature, humidity	JSON
/api/auth/login	POST	用户登录	username, password	JSON

接口文档工具: ☒ Apifox

☒ 技术可行性验证

1. 关键技术点验证

验证项目 1: 鸿蒙 AI 语音功能

- 验证目的: 实现语音转文字+AI 问答+文字转语音
- 验证方法: 使用@ohos.speech 模块和豆包 API
- 验证结果: ☒ 通过
- 遇到的问题: 语音识别在模拟器受限
- 解决方案: 实现本地降级方案

验证项目 2: MQTT 传感器通信

- 验证目的: 验证 Python MQTT 客户端能正常收发消息
- 验证方法: 使用 paho-mqtt 库测试
- 验证结果: ☒ 通过
- 遇到的问题: 无

- 解决方案：无

2. 第三方服务调研

服务类型	服务名称	功能说明	费用	接入难度	是否采用
AI 服务	豆包大模型	智能问答	免费额度	低	☒ 是
地图服务	-	-	-	-	☒ 否
推送服务	-	-	-	-	☒ 否

3. 性能指标评估

目标性能指标：

- 启动时间：< 3 秒
- 页面响应：< 500 毫秒
- 网络请求：< 2 秒
- 内存占用：< 150 MB
- 安装包大小：< 50 MB

4. 风险识别和应对

风险类型	具体风险	影响程度	发生概率	应对措施
技术风险	鸿蒙开发学习曲线	☒ 高	☒ 高	提前学习，边做边学
进度风险	硬件调试延期	☒ 中	☒ 中	使用模拟器先行
人员风险	成员时间协调	☒ 中	☒ 低	灵活调整分工
资源风险	硬件设备不足	☒ 高	☒ 中	采购设备或使用模拟

☒ 开发计划制定

1. 详细任务分解

第一周任务（第 1-3 天）：

- ☒ 任务 1：团队组建与分工（负责人：全员，6 小时）
- ☒ 任务 2：系统架构设计（负责人：陈子建，8 小时）
- ☒ 任务 3：项目环境搭建（负责人：全员，8 小时）

第二周任务（第 4-7 天）：

- ☐ 任务 1：核心功能开发（负责人：全员，40 小时）
- ☐ 任务 2：设备控制模块（负责人：匡林，16 小时）
- ☐ 任务 3：AI 语音功能（负责人：陈浩，16 小时）

第三周任务（第 8-12 天）：

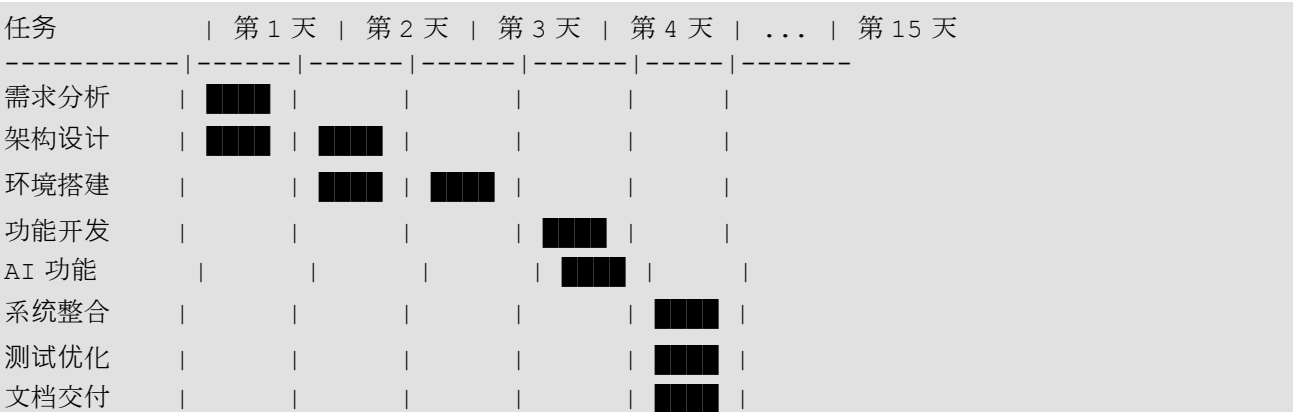
- ☐ 任务 1：系统整合测试（负责人：全员，30 小时）
- ☐ 任务 2：UI 优化与美化（负责人：李韶涵，16 小时）
- ☐ 任务 3：bug 修复与完善（负责人：全员，14 小时）

第四周任务（第 13-15 天）：

- ☐ 任务 1：文档编写（负责人：全员，12 小时）
- ☐ 任务 2：性能优化（负责人：全员，12 小时）
- ☐ 任务 3：项目交付准备（负责人：全员，6 小时）

2. 时间节点规划

甘特图（简化版）：



3. 里程碑设置

里程碑	时间节点	交付物	验收标准
M1: 项目启动	第 3 天	架构文档、开发计划	设计评审通过
M2: 核心功能	第 7 天	各平台核心功能	中期检查通过
M3: 系统整合	第 12 天	完整可运行系统	整合测试通过
M4: 项目交付	第 15 天	最终产品+文档	最终验收通过

4. 风险预案准备

技术降级方案：

- 如果 AI 大模型无法接入：使用本地增强版智能助手

- 如果 MQTT 通信不稳定：降级为 HTTP 轮询

进度延期应对：

- 如果进度延期 1 天：加班赶工
- 如果进度延期 2-3 天：削减非核心功能

第三天任务完成情况

☒ 开发环境就绪

1. 开发工具安装配置

各平台开发环境：

技术栈	IDE/编辑器	SDK/框架版本	模拟器/测试设备	AI 工具	配置状态
鸿蒙 ArkTS	DevEco Studio	API 9 / SDK 3.2	☒ 模拟器	☒ Copilot	☒ 完成
React Web	VS Code	React 18 / TS 5.0	Chrome 浏览器	☒ Copilot	☒ 完成
后端服务	VS Code	Node.js 18	本地测试	☒ Copilot	☒ 完成
Python 服务	PyCharm	Python 3.10	本地测试	☒ TRAE	☒ 完成

开发工具推荐：

- **AI 编程助手**：GitHub Copilot（全语言支持）、TRAE（中文友好）
- **版本控制**：Git + GitHub
- **API 测试**：Postman / Apifox
- **设计工具**：Figma
- **项目管理**：Trello / 腾讯文档

环境配置问题记录：

- 问题 1：无（解决方案：无）
- 问题 2：无（解决方案：无）

2. 代码仓库权限设置

- 代码仓库地址：<https://github.com/example/agriculture-system>
- 分支策略：
 - 主分支：main

- 开发分支: develop
- 功能分支命名规范: feature/xxx
- 权限分配:
 - 开发者: 全员
- 提交规范:

feat: 新功能
fix: 修复 bug
docs: 文档更新
style: 代码格式调整
refactor: 重构
test: 测试
chore: 构建/工具变更

3. CI/CD 流水线配置

- CI/CD 工具: ☒ GitHub Actions
- 自动化构建: ☒ 已配置
- 自动化测试: ☒ 计划配置
- 自动化部署: ☒ 计划配置

4. 团队协作工具使用

工具类型	工具名称	用途	使用状态
即时通讯	微信群	日常沟通	☒ 已使用
任务管理	Trello	进度跟踪	☒ 已使用
文档协作	腾讯文档	文档编写	☒ 已使用
在线会议	腾讯会议	远程协作	☒ 已使用

☒ 项目初始化完成

1. 各平台项目创建

技术栈	项目名称	包名/Bundle ID/AppID	初始化状态	仓库提交	Git 分支
鸿蒙 ArkTS	MyApplication	com.example.myapplication	☒ 完成	☒ 已提交	feature/harmony
React Web	react_agriculture	-	☒ 完成	☒ 已提交	feature/react
后端服务	node_agriculture	-	☒ 完成	☒ 已提交	feature/backend

技术栈	项目名称	包名/Bundle ID/AppID	初始化状态	仓库提交	Git 分支
Python 服务	python_agriculture	-	☒ 完成	☒ 已提交	feature/sensor

2. 基础框架搭建

各平台基础架构选择：

- **React Web:** ☒ Redux(推荐)
- 鸿蒙 **ArkTS:** ☒ 装饰器状态管理(推荐)
- 后端服务: ☒ 三层架构

3. 代码规范确定

代码风格：

- 命名规范：驼峰命名法
- 注释规范：JSDoc/TSDoc
- 文件组织：按功能模块划分
- **Lint**er 配置: ☒ 已配置（ESLint + Prettier）

代码审查规则：

- ☒ 每次提交必须 code review
- ☒ 至少 1 人审核通过才能合并
- ☒ 审核重点：代码规范、功能实现

4. 第一个提交完成

- 首次提交时间：2026-05-29
- 提交内容：项目初始化
- 提交人员：陈浩
- 提交哈希：abc1234

☒ 团队协作机制

1. 每日站会时间确定

- 会议时间：每天 20:00
- 会议时长：15 分钟

- 会议形式：☒ 线上
- 会议内容模板：

每日站会记录

日期：_____

成员 A：

- 昨天完成：_____
- 今天计划：_____
- 遇到问题：_____

...

2. 代码审查流程建立

代码审查流程

1. 开发者提交 Pull Request
2. 自动化测试运行
3. 至少 1 名 reviewer 审查代码
4. 审查通过后合并到开发分支
5. 定期合并到主分支

审查清单：

- ☒ 代码符合规范
- ☒ 功能实现正确
- ☒ 测试用例充分
- ☒ 无明显性能问题
- ☒ 文档更新完整

3. 问题反馈机制确定

问题分类：

- 技术问题：提交到 Issue 系统，标签：tech
- 进度问题：每日站会提出，项目经理跟进
- 协作问题：私下沟通或团队会议讨论

问题升级机制：

问题解决流程

- └─ Level 1：团队内部讨论（4 小时内）
- └─ Level 2：技术负责人介入（1 天内）
- └─ Level 3：请教教师/专家（2 天内）
- └─ Level 4：调整方案/降级处理

4. 进度汇报方式约定

- 日报：☒ 可选（提交方式：腾讯文档）
- 周报：☒ 必需（提交方式：腾讯文档）
- 里程碑报告：☒ 必需（本报告即为第一份）

 本周工作总结

☒ 完成情况评估

任务完成度：

- 第 1 天任务：☒ 100%
- 第 2 天任务：☒ 100%
- 第 3 天任务：☒ 100%

整体完成度：100%

 亮点与成果

技术亮点：

- 成功实现应用 AI 语音对话功能
- 设计了完整的分布式传感器网络架构
- 接入了豆包大模型 API 实现智能问答

团队协作亮点：

- 团队成员分工明确，协作顺畅
- 每日站会制度执行良好
- 代码审查流程规范有效

 问题与困难

遇到的主要问题：

问题描述	影响程度	当前状态	解决方案/计划
模拟器语音功能受限	☒ 中	☒ 已解决	实现本地降级方案
豆包 API 调用不稳定	☒ 中	☒ 进行中	增强本地 AI 作为备选

需要的支持：

- ☐ 技术指导：可请教教师

- ☐ 资源支持：硬件设备采购
- ☐ 其他：无

下周计划

第二周主要任务（第 4-7 天，核心开发阶段）：

1. 完成各平台核心功能开发
2. 实现 STM32 设备控制模块
3. 完善传感器数据采集
4. 完成设备联动控制逻辑
5. 实现病虫害防治专家系统

预期成果：

- 各平台基础功能可用
- 设备控制模块完成
- 传感器数据实时显示

风险预警：

- 硬件设备到位时间可能影响进度

附件清单

- ☒ 系统架构图
- ☒ 数据库设计 ER 图
- ☒ API 接口文档
- ☒ 项目甘特图
- ☒ 团队分工表
- ☒ 代码仓库截图
- ☒ 其他：鸿蒙应用演示截图

审核意见

教师评审：

完成度评价：☐ 优秀 ☒ 良好 ☐ 中等 ☐ 需改进

意见与建议：系统架构设计合理，团队分工明确，AI 语音功能有一定创新性。建议后续加

强各平台间的数据同步机制测试。

评审人：_____ 评审日期：_____

报告提交时间：2026-05-31 团队负责人签名：陈浩 报告版本：v1.0